

# ROTEIRO DO VÍDEO PITCH

## Loja Veloz · Cloud DevOps Engineer · 2025

Entrega Contínua de Microsserviços: Docker Compose → Kubernetes

**Como usar este roteiro:** cada seção indica o slide correspondente, o tempo de fala e o texto sugerido (em itálico). Adapte as falas ao seu estilo. Gravação recomendada: OBS Studio ou Loom com compartilhamento de tela (slides) + webcam.

■ Duração total alvo: **3 min 40 s – 4 min**

### 1 ABERTURA & CONTEXTO

~30 s

Slide 1 – Capa | Slide 2 – Problema

0:00 → 0:30

“*Olá! Meu nome é [seu nome] e este é o pitch técnico do projeto Loja Veloz. Vou mostrar como modernizamos a plataforma de pedidos de uma empresa de varejo digital que sofria com indisponibilidades em deploys, dificuldade de escalar no pico de tráfego e baixa rastreabilidade de falhas.*”

■ Apareça na webcam nos primeiros 5 segundos para criar conexão com o avaliador.

“*O desafio era claro: precisávamos de um ambiente de desenvolvimento padronizado, containerização segura, orquestração em Kubernetes, um pipeline de CI/CD automatizado e observabilidade completa — tudo em 4 semanas, antes de uma campanha promocional.*”

■ Avance para o slide 2 ao mencionar os problemas. Aponte para os cards de problema.

### 2 VISÃO GERAL DA ARQUITETURA

~30 s

Slide 3 – Arquitetura

0:30 → 1:00

“*A plataforma Pedidos Veloz é composta por cinco microsserviços: API Gateway, Serviço de Pedidos, Pagamentos, Estoque e um banco PostgreSQL. A comunicação entre Pedidos e Estoque é assíncrona via RabbitMQ — quando um pedido é criado, o evento é publicado no broker, desacoplando os serviços e garantindo resiliência mesmo que o Estoque esteja temporariamente indisponível.*”

■ Aponte para o diagrama de arquitetura enquanto fala de cada serviço.

“*Todos os serviços são instrumentados com OpenTelemetry e monitorados por Prometheus, Grafana e Jaeger — eu mostro isso em detalhes mais à frente.*”

### 3 CONTAINERIZAÇÃO — Docker e Compose

~40 s

“ Para o ambiente local, usamos Docker Compose. Com um único comando — `docker compose up --build` — qualquer desenvolvedor sobe todos os serviços, redes, volumes e variáveis de ambiente de forma idêntica, eliminando o clássico 'funciona na minha máquina'. ”

■ Avance para o slide 4. Mostre o código do Compose enquanto fala.

“ Cada serviço tem seu próprio Dockerfile com build multi-stage: o primeiro estágio compila e instala dependências; o segundo estágio copia apenas o artefato final para uma imagem Alpine mínima. O resultado é uma imagem 80% menor, sem ferramentas de desenvolvimento, executando com usuário não-root. Segurança desde o início. ”

■ Avance para o slide 5. Destaque as setas entre os estágios do build.

## 4 KUBERNETES — Produção

~35 s

“ Para produção, usamos Kubernetes. Cada serviço tem um Deployment com 3 réplicas mínimas, configurado com readiness e liveness probes — o Kubernetes só envia tráfego para pods saudáveis e reinicia automaticamente qualquer pod com falha. ”

“ Configurações ficam em ConfigMaps e credenciais em Secrets, integrados ao HashiCorp Vault via External Secrets Operator — nenhuma senha fica hardcoded. Aplicamos Pod Security Admission com perfil 'restricted' e Network Policies para isolar os namespaces. O Horizontal Pod Autoscaler escala de 3 até 15 pods automaticamente com base em 60% de uso de CPU. ”

■ Avance para o slide 6. Aponte para as três colunas: Deploy, Config e HPA.

## 5 PIPELINE CI/CD

~30 s

“ O pipeline é automatizado com GitHub Actions. A cada push ou pull request, executamos: lint, testes com cobertura mínima de 80%, build da imagem Docker, scan de vulnerabilidades com Trivy — se houver CVE crítica, o pipeline falha e não avança. ”

“ Após aprovação, a imagem é publicada no GitHub Container Registry com tag semântica. O deploy usa `kubectl` com rolling update e valida o rollout — se falhar, o rollback é executado automaticamente em segundos. Nenhuma credencial aparece em log: tudo gerenciado via GitHub Secrets. ”

■ Aponte para o fluxo de setas entre os estágios do pipeline no slide 7.

## 6 OBSERVABILIDADE & DEPLOY

~30 s

“ A observabilidade tem três pilares. Métricas: Prometheus coleta dados de todos os serviços e o Grafana exibe dashboards com SLOs — latência p95 abaixo de 300ms e uptime acima de 99,5%. Logs: Fluentd centraliza logs JSON estruturados no Loki com correlação por trace ID.

*Traces: OpenTelemetry instrumenta cada serviço e o Jaeger mostra a jornada completa de uma requisição por todos os microsserviços. ”*

■ Avance para slide 8. Aponte para cada coluna: Métricas, Logs, Traces.

*“ Para deploy, usamos Rolling Update com maxUnavailable zero — sempre há réplicas saudáveis atendendo. Em caso de falha nas probes, rollback automático é acionado. Para a campanha promocional, o HPA garante escalonamento automático, e podemos elevar o mínimo de réplicas manualmente para absorver picos previstos. ”*

■ Avance para slide 9. Mostre o gráfico de barras com o HPA escalando.

## 7 TERRAFORM & ENCERRAMENTO

~30 s

Slides 10, 11 e 12 – Terraform · Plano · Encerramento

3:15 → 3:55

*“ Toda a infraestrutura cloud é provisionada com Terraform — cluster GKE, VPC, Artifact Registry e banco Cloud SQL. O estado é versionado em bucket GCS e qualquer mudança passa por pull request com terraform plan antes de ser aplicada. ”*

■ Avance para slide 10. Depois para slide 11 ao falar do plano.

*“ O plano de execução é de quatro semanas: na primeira, fundação com Compose e Dockerfiles. Na segunda, manifests Kubernetes e cluster de staging. Na terceira, pipeline CI/CD completo. Na quarta, observabilidade, load test com k6 e pré-escalonamento para a campanha. ”*

*“ O resultado é uma plataforma cloud-native completa: desenvolvimento padronizado, deploys sem risco, escalonamento automático e visibilidade total em produção. O código, os manifests e este relatório estão no repositório — link no README. Obrigado! ”*

■ Avance para o slide 12 – Encerramento. Olhe para a câmera nos últimos 5 segundos.

---

### Dicas para a Gravação

- **Ferramenta gratuita:** OBS Studio (obs.project.org) — capture a tela com os slides abertos no PowerPoint/LibreOffice e ative a webcam no canto inferior direito.
- **Resolução:** grave em 1920×1080 (Full HD) e exporte em MP4 H.264 para upload no YouTube.
- **Ritmo:** fale devagar, especialmente nos slides técnicos. 4 minutos é fácil de preencher — não precisa correr.
- **Ensaio:** grave um teste de 30 segundos antes para verificar áudio, iluminação e enquadramento.
- **Upload:** no YouTube, pode ser 'Não listado' (unlisted) — só quem tiver o link acessa. Copie o link para o README e para o PDF do relatório.
- **Slides:** use modo apresentação (F5 no PowerPoint) e avance manualmente conforme o roteiro.